Skreen.ai — Technical Assessment: Full-Stack Developer (AI Engineering)

Thanks for taking the time to work through this. This exercise is designed to reflect the kind of work you'd actually do at Skreen in your first few weeks: building a small AI-powered feature end-to-end, from UI through backend to an LLM call, using tools of your own choosing.

There's no single "correct" implementation — we're more interested in how you think and the choices you make than in matching a checklist.

Time expectation

You have **1 week** from receiving this to submit, so you can fit it around a current job if needed. There's no fixed hour-count expectation — a few focused hours is the target. Please don't over-invest; extra polish beyond the core deliverable isn't required and won't earn extra credit.

What you'll build: "AI-Assisted Candidate Screener"

A small full-stack app that scores candidates against a job, using an LLM, and proposes a follow-up screening question.

Use whatever stack you're most comfortable and productive in — any modern frontend framework, any Postgres-backed backend (a serverless/edge function, a small API server, whatever you prefer), and any LLM provider (OpenAI, Anthropic, or similar).

1. Data model

A Postgres schema (2–3 tables) for: - `jobs` — title + free-text requirements. - `candidates` — name + free-text CV/resume content.

No file upload or PDF parsing is required — plain text is fine.

2. LLM scoring feature

Given a job's requirements and a candidate's CV text, call an LLM to produce a **structured** match result, for example:

```
{
 "overall_score": 0-100,
 "matched_requirements": ["..."],
 "gaps": ["..."],
 "rationale": "..."
}
```

Use function-calling / structured-output / JSON-schema style output — not just a free-text prompt you hope parses cleanly.

3. UI

A simple page to: - Create/view a job. - Load candidate CVs (provided — see "Test data" below). - Trigger scoring and see results in a ranked list with the structured breakdown.

4. Backend/API layer

A serverless/edge function or a small API route, your choice.

5. Tests

At least one meaningful automated test. For example: that the LLM response is validated/coerced into the expected schema, and that malformed or partial LLM output is handled gracefully rather than crashing the app.

6. Follow-up screening question

After scoring a candidate, have the app propose one or more follow-up screening question(s) for a recruiter to ask that candidate, based on the gaps identified in scoring.

We're intentionally not specifying how many questions, what makes a good one, or how they should be generated — that's for you to decide. Be ready to talk through your reasoning in the follow-up call.

Voice is **not** required for this step — text in, text out is enough. If you want to take it further, see the stretch goal below.

Test data (use exactly as provided — don't substitute your own)

Use the same job and candidates for everyone taking this assessment, so results are comparable: - `Assessment_Job_Spec_FixedData.md` — the job to score against. - `Assessment_CV_A_StrongMatch.md` - `Assessment_CV_B_AmbiguousMatch.md` - `Assessment_CV_C_WeakMatch.md`

Stretch goal (optional — pick this up only if you have time and want to go further)

Turn the follow-up question step into an actual **voice interview**: use any speech-to-text/text-to-speech provider (ElevenLabs, Deepgram, or even a browser speech API as a fallback) to have the app *ask* the question aloud, capture a spoken answer, transcribe it, and feed that back into the flow.

If you want to push further still: make it **agentic rather than single-shot** — instead of always asking exactly one question, have the app decide *whether* a follow-up is even needed, and if the transcribed answer still leaves a gap unresolved, ask one more (capped at 2 rounds), reasoning over its own prior output rather than following a fixed script.

This is a genuine stretch goal — a partial or incomplete attempt is still a positive signal, and skipping it entirely is completely fine.

What to submit

- **A ZIP file** containing your source code (no need to set up a GitHub repo).
- **A `README.md`** at the root covering:
- How to run the project locally (install/build/start commands).
- Any environment variables required to run it (e.g. LLM API key, database connection string) — include a `.env.example` listing the variable names only, NO REAL SECRETS COMMITTED.
- Which provider(s) you used (LLM, database, and voice provider if you attempted the stretch goal), so we know what to configure before running it.
- Any known limitations or unfinished pieces, in a line or two.

No written trade-offs essay or video walkthrough needed — we'll cover that in the follow-up conversation.

What happens next

After you submit, we'll schedule a 30–45 minute call to walk through your code together and ask a few follow-up questions about the choices you made.

Good luck — and thank you again for the time you're putting into this.